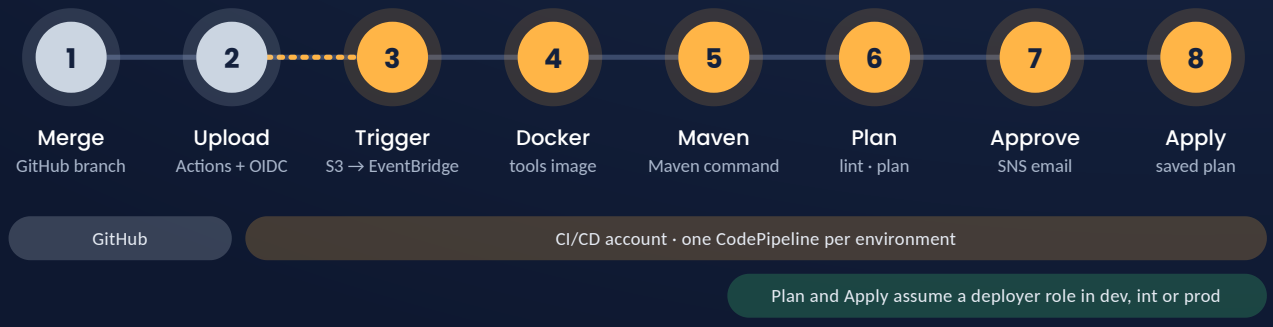
BUILD GUIDE · INFRASTRUCTURE AS CODE

Terraform CI/CD on AWS with a GitHub trigger

How a merge to `dev`, `int` or `prod` becomes a reviewed, approved Terraform apply, and how to build the same pipeline for your own project.



3
environments: dev, int and prod

5
pipeline stages, with a manual approval before apply

1 → 3
one CI/CD account, one target account per environment

0
long-lived AWS keys stored anywhere

PLACEHOLDERS USED

`<project>`	your resource name prefix
`ENV`	`dev`, `int` or `prod`
`<owner>/<repo>`	your GitHub repository
`<cicd-account-id>`	the account that runs the pipelines

COLOUR KEY

- GitHub and people
- AWS services
- IAM roles
- 1 Step of one run (1 to 8)

IN THIS GUIDE

01 Summary · 02 Architecture · 03 Pipeline flow · 04 IAM roles · 05 Build it from scratch · 06 Problems and fixes · 07 Trade-offs

Account IDs, bucket names, email addresses and tokens are left out on purpose.

October 2026

BEFORE YOU START

00 How to use this guide

This guide is for an engineer who wants to build the same pipeline for their own project. It assumes you know Terraform and AWS. You do not need to have seen this design before.

01 **Summary**
What the setup does and why it is built this way

02 **Architecture**
The full flow from a merge to a Terraform apply

03 **Pipeline flow**
Each step of one run, as a table and a sequence

04 **IAM roles**
Who assumes which role, trust conditions and permissions

05 **Build it from scratch**
Nine steps, from prerequisites to promotion

06 **Problems and fixes**
Errors met while building this, and how they were fixed

07 **Trade-offs**
What the design gains and costs, and open points to check

Reading the diagrams

Colours mean the same thing everywhere: grey is GitHub or a person, orange is an AWS service and green is an IAM role. Dark numbered circles 1 to 8 are the steps of one run and match the table in section 3 and Figures 2 and 3.

Safe to share

Names use placeholders such as `<project>` and `ENV`. There are no account IDs, bucket names, email addresses or tokens in this document.

What this is based on

The working setup's notes and diagrams. Where the notes do not say something, this guide says so instead of guessing. Section 7 lists the open points to check in your repository.

Terms used in this guide

OIDC (OpenID Connect)
GitHub gives each workflow run a short-lived signed token. AWS checks the token and returns temporary credentials for one role. No AWS keys are stored in GitHub.

AssumeRole
An AWS identity asks STS for temporary credentials of another role. The role's trust policy decides who may ask.

Trust policy and permissions
The trust policy says who may assume a role. The permissions policy says what the role may do once assumed.

Saved plan
`terraform plan -out` writes the planned changes to a file. Applying that file makes exactly those changes, nothing else.

Destroy guard
A check in the Plan stage. The build fails if the plan would destroy any resource.

Detailed exit code
`terraform plan -detailed-exitcode` returns 0 for no changes, 1 for an error and 2 when changes are pending.

Tools image
A Docker image with Terraform, TFLint, AWS CLI, jq and Maven, stored in ECR under a pinned tag.

Promotion
Moving the same code from dev to int to prod by merging it from one branch into the next.

SUMMARY

01 What it does and why

A merge to the `dev`, `int` or `prod` branch zips the repository and uploads it to an S3 bucket. The upload starts the AWS CodePipeline for that environment. The pipeline builds a tools image, runs Maven, runs `terraform plan`, waits for a person to approve, and then applies the saved plan.

All three pipelines live in one AWS account, the **CI/CD account**. Each pipeline assumes a deployer role in its own target account to plan and apply. GitHub never holds credentials that can change infrastructure: its only permission is to upload the source zip.

Why it is built this way

No stored AWS keys

GitHub gets temporary credentials through OIDC. Every other step assumes an IAM role. Nothing long-lived is kept in GitHub or in the pipeline.

One pipeline per environment

Each branch uploads to its own key, `ENV/source.zip`. The EventBridge rule for that key starts only that environment's pipeline.

Least access per role

GitHub's role can only upload its own `source.zip`. Each build role can assume only its own environment's deployer role.

A person approves every apply

The pipeline stops after Plan and sends an SNS email. A reviewer reads the plan and approves or rejects it in the console.

Apply what was reviewed

Plan fails if anything would be destroyed. Apply uses the saved plan, then runs a fresh plan that must show no changes.

Pinned tools

The Docker stage builds one tools image (Terraform, TFLint, AWS CLI, jq, Maven) and pushes it to ECR with a pinned, immutable tag.

The accounts involved

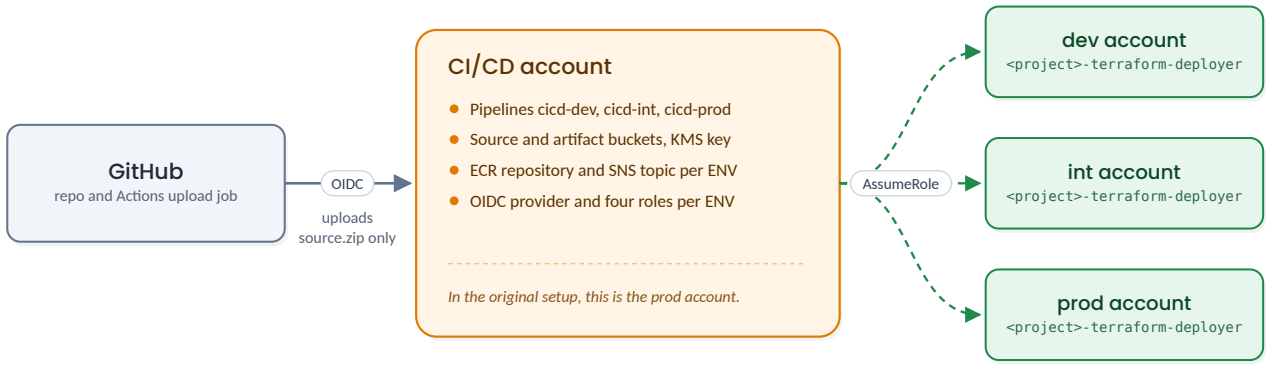


Figure 1 Account view. GitHub can only upload the source zip. Every infrastructure change is made from the CI/CD account through the deployer role in the target account.

Where the CI/CD account lives

In the original setup the pipelines run in the prod account. A dedicated tooling account isolates them better. Section 7 covers this trade-off.

ARCHITECTURE

02 From a merge to a Terraform apply

Figure 2 shows one run for one environment. `ENV` is `dev`, `int` or `prod`. Read it top to bottom: GitHub, then the CI/CD account, then the target account.

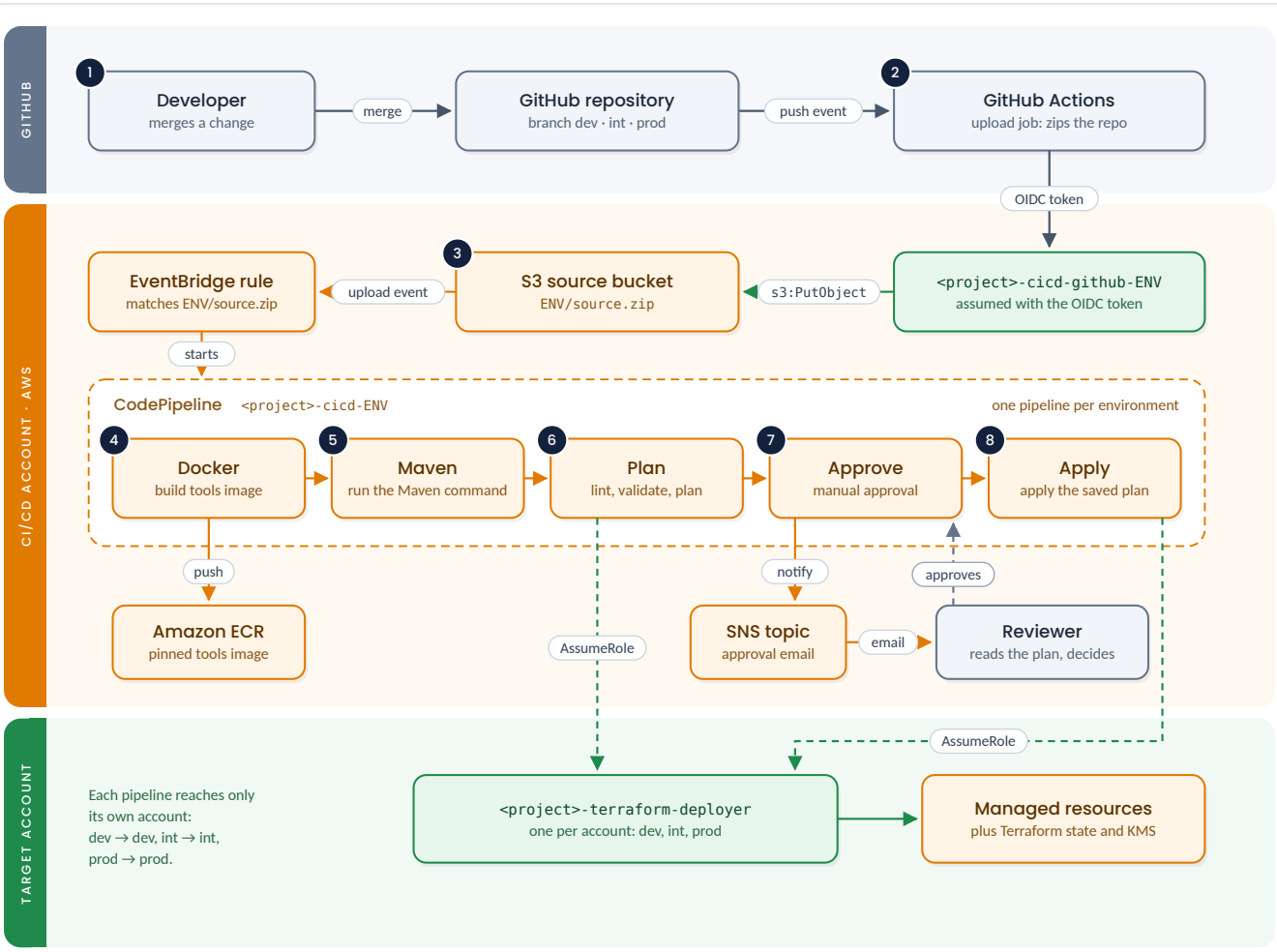


Figure 2 End-to-end architecture. Numbers 1 to 8 match the steps in section 3. The Plan and Apply stages both assume the deployer role in the target account.

GitHub or people · AWS services · IAM roles · 1 step number · dashed green arrow: role assumption

Lane	What lives there
GitHub	The repository with `dev`, `int` and `prod` branches, and the Actions workflow that uploads the source zip. The original setup uses GitHub Enterprise Server.
CI/CD account	The OIDC provider, one KMS key, the source and artifact buckets, and per environment: an EventBridge rule, a pipeline, its CodeBuild projects, an ECR repository, an SNS topic and four roles.
Target accounts	One `<project>-terraform-deployer` role in each of the dev, int and prod accounts, with access to the Terraform state bucket and the resources Terraform manages.

PIPELINE FLOW

03 One run, step by step

Every run follows the same eight steps. Steps 4 to 8 are CodePipeline stages, each run by a CodeBuild project, except Approve, which is a manual approval action.

#	Actor	What happens
1	Developer	Merges a change into `dev`, `int` or `prod`.
2	GitHub Actions	Assumes `<project>-cicd-github-ENV` through OIDC, zips the repo and uploads `ENV/source.zip` to the source bucket.
3	S3 + EventBridge	The upload event starts the pipeline for that environment only.
4	Docker stage	Builds the tools image (Terraform, TFLint, AWS CLI, jq, Maven) and pushes it to ECR with a pinned tag. If the tag already exists, it skips the build.
5	Maven stage	Runs the Maven command inside the tools image.
6	Plan stage	Assumes the deployer role in the target account. Runs `fmt -check`, `tflint`, `init`, `validate` and `plan`. Fails if the plan would destroy anything.
7	Approve stage	Sends an SNS email. A reviewer reads the plan and approves or rejects in the console.
8	Apply stage	Applies the exact saved plan, then runs a post-apply plan that must exit 0.

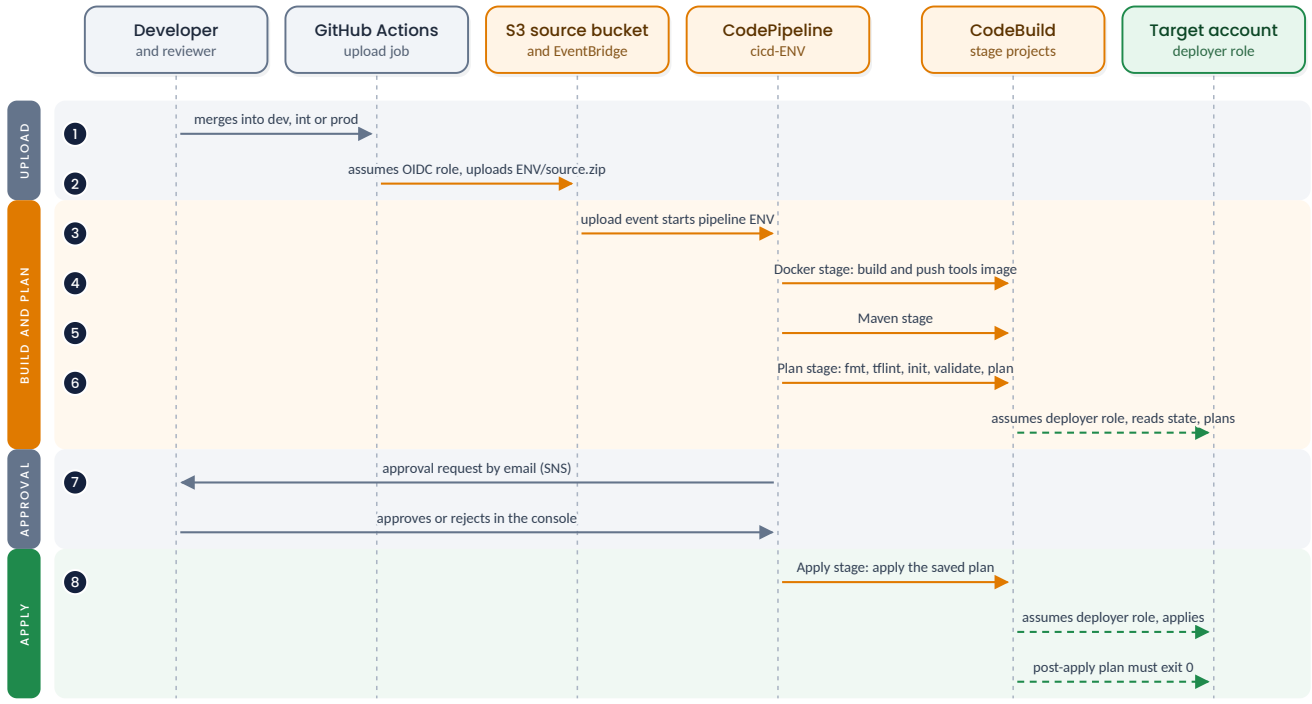


Figure 3 The same run as a sequence. Dashed green arrows are calls made with the deployer role's temporary credentials. If the reviewer rejects, nothing is applied.

IAM ROLES

04 Who assumes whom

There are five role types. Four live in the CI/CD account and exist once per environment. The deployer role lives in each target account. Each role trusts exactly one kind of principal.

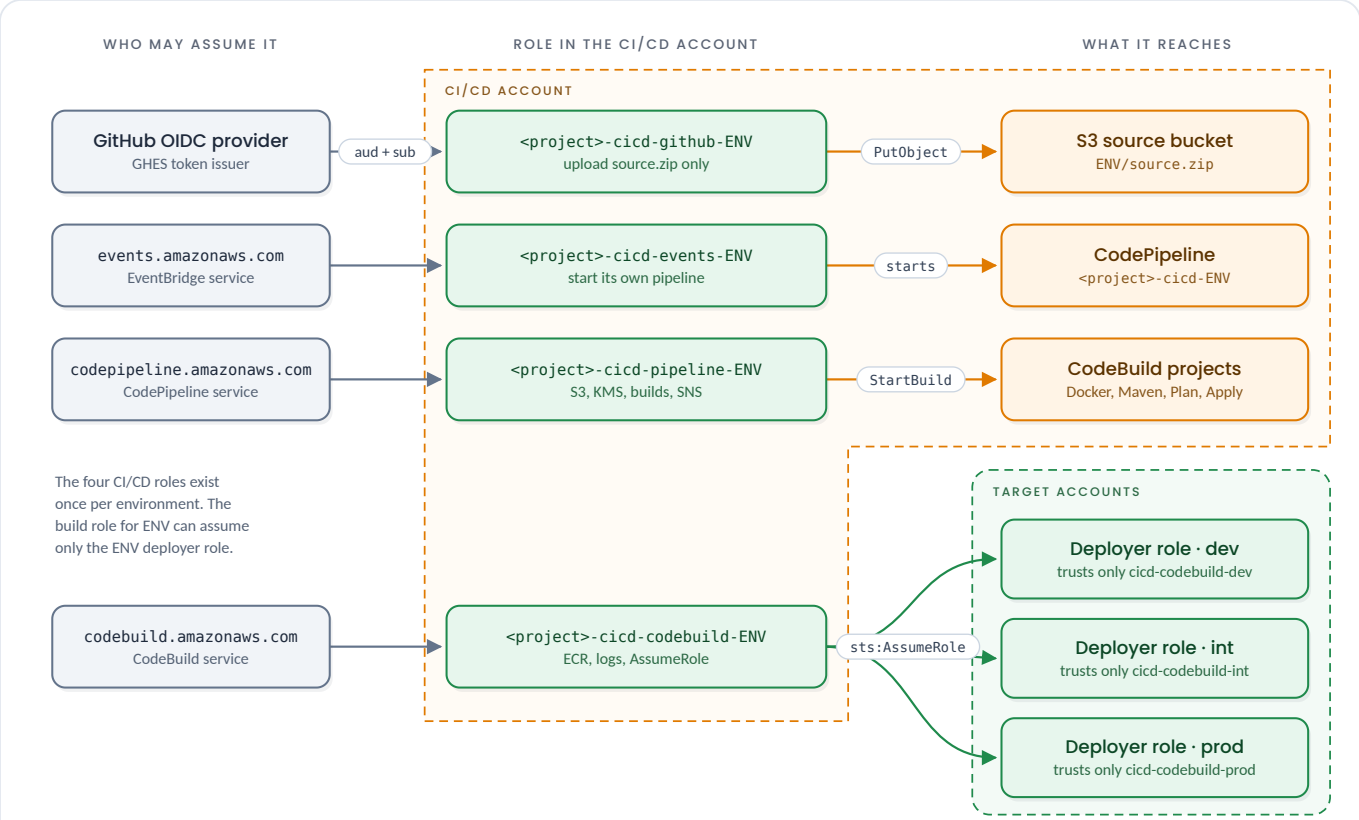


Figure 4 Trust relationships. Left: who may assume the role. Middle: the role. Right: what the role can reach. Each build role may assume only its own environment's deployer role.

Design rules

No long-lived keys

The upload uses OIDC. Everything else uses role assumption, so every credential is temporary.

One role per branch

The OIDC trust matches the branch in `sub`, so the `dev` branch can assume only `<project>-cicd-github-dev`, and so on.

Exact deployer trust

The deployer trusts the CI/CD account only when `aws:PrincipalArn` is the matching build role ARN. No other principal in that account can assume it.

Every role, its trust and its permissions

The table lists each role type once. Replace `ENV` with `dev`, `int` or `prod` to get the real role names.

Role	Account	Trusted principal	Trust condition	Main permissions
`<project>-cicd-github-ENV`	CI/CD	GitHub OIDC provider (GHES token issuer)	`aud` = `sts.amazonaws.com` `sub` = `repo:<owner>/<repo>:ref:refs/heads/ENV`	`s3:PutObject` on `ENV/source.zip`; KMS `GenerateDataKey` and `Decrypt` on the CI/CD key
`<project>-cicd-events-ENV`	CI/CD	EventBridge service `events.amazonaws.com`	Service principal; see note below	`codepipeline:StartPipelineExecution` on its own pipeline
`<project>-cicd-pipeline-ENV`	CI/CD	CodePipeline service `codepipeline.amazonaws.com`	Service principal; see note below	Read the source object, read and write artifacts, KMS on the CI/CD key, start and read CodeBuild projects, `sns:Publish` to its approval topic
`<project>-cicd-codebuild-ENV`	CI/CD	CodeBuild service `codebuild.amazonaws.com`	Service principal; see note below	Write logs, read and write artifacts, KMS, ECR push and pull on its own repository, `sts:AssumeRole` on its environment's deployer role
`<project>-terraform-deployer`	Each target account (dev, int, prod)	The CI/CD account	`aws:PrincipalArn` = the `<project>-cicd-codebuild-ENV` role ARN	The Terraform state bucket and its KMS key, plus the specific resources Terraform manages

Trust conditions on the service roles

The source notes name only the service principal for the events, pipeline and CodeBuild roles. Check the trust policy JSON in your `iam` module before you copy it. If it has no `aws:SourceAccount` or `aws:SourceArn` condition, consider adding one.

Reading a role name

`<project>-cicd-codebuild-dev` is the build role of the dev pipeline. It lives in the CI/CD account and may assume only `<project>-terraform-deployer` in the dev account.

BUILD IT FROM SCRATCH

05 Nine steps

Build in this order. Steps 2 and 3 are Terraform you apply by hand once. After that, every change goes through the pipeline.

1 Prerequisites

- Decide which account holds the CI/CD. A dedicated tooling account is better than a workload account.
- Confirm that GitHub can issue OIDC tokens and that AWS can reach the token issuer. Without this the OIDC provider will not work.
- Set up an S3 backend for Terraform state, and AWS SSO profiles for each account.

2 CI/CD stack applied in the CI/CD account · cicd/ in the original repo

Split the stack into modules:

Module	What it creates
`kms`	One key for the buckets, logs and artifacts, with a key policy.
`s3`	The source bucket and the artifact bucket: encrypted, versioned, TLS-only, public access blocked, with an event notification on upload.
`ecr`	One tools-image repository per environment: immutable tags, scan on push, KMS encryption.
`sns`	One approval topic per environment, with email subscriptions.
`iam`	The GitHub OIDC provider and the four CI/CD role types, with their JSON policy files.
`codebuild`	The Docker, Maven, Plan and Apply projects for each environment, with their buildspecs.
`pipeline`	One pipeline per environment, plus the EventBridge rule on `ENV/source.zip`.

3 Deployer roles applied once in each target account · cicd/deployer/

Create `<project>-terraform-deployer` in the dev, int and prod accounts. Each one trusts the CI/CD account, restricted with `aws:PrincipalArn` to that environment's `<project>-cicd-codebuild-ENV` role. Give it the state bucket, its KMS key and the resources Terraform manages, nothing more.

4 Tools image docker/

Write a Dockerfile with Terraform, TFLint, AWS CLI, jq and Maven. The Docker stage builds it and pushes the tag set in `tools_image_tag`.

- ECR tags are immutable, so **bump `tools_image_tag` whenever the Dockerfile changes**. Otherwise the stage sees the old tag and skips the build.
- Pin the Ubuntu base image to the previous release and run as a non-root `USER` (see section 6).

5 GitHub workflow

Trigger on a push to `dev`, `int` or `prod`, filtered to the infrastructure path. The job assumes the matching OIDC role, zips the repo and uploads `<branch>/source.zip`. A `workflow_dispatch` trigger lets you run it by hand when the path filter does not match.

6 Repository variables

Two variables: the CI/CD account ID and the source bucket name. No AWS keys are stored in GitHub.

7 Pipeline stages and buildspecs

one CodeBuild buildspec per stage

Docker tag exists in ECR? › yes: skip › no: build tools image › push pinned tag

Maven run the Maven command inside the tools image

The notes do not name the Maven goal. Read it from the Maven buildspec in your repository.

Plan assume deployer role › `fmt -check` › `tflint` › `init` › `validate` › plan, saved › destroy guard

The build fails on unformatted files, lint errors, invalid configuration, or any planned destroy.

Approve SNS email › reviewer reads the plan › approve or reject in the console

Apply assume deployer role › apply the saved plan › post-apply `plan -detailed-exitcode` › must exit 0

Exit code 0 proves that the applied state matches the code and nothing is left to change.

8 Local checks

pre-push hook

A pre-push hook runs `terraform fmt`, `tflint --fix` and `terraform validate` on each Terraform root, so failures show up before the pipeline runs.

9 Promotion

dev → int → prod

Test on dev, then merge the same code into int, then into prod. Before each merge, run a local `terraform plan` against the next environment. Read the Plan stage output before you approve.



Figure 5 Promotion order. The same three checks happen before every apply.

PROBLEMS AND FIXES

06 What went wrong while building it

All seven of these came up while building the original setup and were fixed as shown. Most are caught earlier if the pre-push hook runs.

Where	Symptom	Cause	Fix
Docker stage	`tar: Function not implemented`	The newest Ubuntu base image does not work with CodeBuild's Docker.	Pin the base image to the previous Ubuntu release.
Plan stage	`terraform fmt -check` fails	Files were pushed without being formatted.	Format locally first. The pre-push hook does this.
Plan and local	TFLint warnings on modules	Modules without version constraints.	Add `versions.tf` with `required_version` and a provider version to every module.
Apply stage	Post-apply plan not clean with no real changes	A data source inside a `check` block is deferred to apply, so `plan -detailed-exitcode` returns 2.	Declare the data source outside the `check` block.
Apply stage	Saved plan is stale	Two pipeline executions overlapped.	Start a fresh execution and approve only the newest one.
GitHub Actions	Workflow did not start	The path filter did not match, or the workflow file is not on that branch.	Fix the filter or add the file to the branch. Run it by hand with `workflow_dispatch`.
Repository scan	IaC scanner findings	Findings on the ECR repository and the Dockerfile.	Use immutable ECR tags and a non-root `USER` in the Dockerfile.

Habits that avoid repeats

Run the pre-push hook before every push. Approve only the newest pipeline execution. When you change the Dockerfile, bump `tools_image_tag` in the same commit.

TRADE-OFFS

07 What it gains and what it costs

The design keeps every credential short-lived and inside AWS. The price is a second CI system to run next to GitHub Actions.

Gains

- Everything runs inside AWS, and every credential is short-lived.
- GitHub can only upload a zip. It cannot change infrastructure directly.
- Each environment has its own pipeline, roles and approval topic.
- A person approves every apply, and Apply changes only what was planned.

Costs

- Two systems to maintain: GitHub Actions and CodePipeline.
- The plan is in the build logs, not on the pull request. A read-only plan job in Actions could add that.
- Hosting the CI/CD in a workload account (here prod) is simpler, but a dedicated tooling account isolates it better.

Compared with doing everything in GitHub Actions

A general comparison of the two approaches. It was not measured in this project.

	This design	Everything in GitHub Actions
Where Terraform runs	CodeBuild, inside AWS	A GitHub-hosted or self-hosted runner
What GitHub can do in AWS	Upload `ENV/source.zip`, nothing else	Assume roles that change infrastructure
Credentials	OIDC for the upload, role assumption for the rest	OIDC from the runner to a role in each target account
Manual approval	CodePipeline approval action with SNS email	GitHub environment protection rules, where your GitHub plan supports them
Where the plan is shown	CodeBuild logs	Can be posted on the pull request
Systems to maintain	Two: GitHub Actions and CodePipeline	One

Open points to check before you copy

The source notes do not cover these. Read them from the repository, do not assume them.

- The exact Maven command the Maven stage runs.
- How the destroy guard reads the plan in the Plan buildspec.
- The trust conditions on the events, pipeline and CodeBuild roles.
- The Terraform, TFLint and provider versions pinned in the tools image and in `versions.tf`.